TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ
TRUYỀN THÔNG VIỆT – HÀN
KHOA KHOA HỌC MÁY TÍNH

---- ----

VKU

ĐỒ ÁN MÔN HỌC:
LINUX VÀ PHẦN MỀM NGUỒN MỞ

ĐỀ TÀI: Configure and Deploy MLOps (Machine Learning Ops) Services on Arch Linux & Ubuntu

Sinh viên thực hiện : Nguyễn Công Đức (Trưởng nhóm - Arch Linux)

Nguyễn Tùng Dương (Thành viên - Ubuntu Infra)

Thành viên 3 (Thành viên - Observability & Benchmark)

Giảng viên hướng dẫn : TS. Hoàng Hữu Đức

Đà Nẵng, tháng 10 năm 2026

LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành đến TS. Hoàng Hữu Đức – người thầy đã tận tình hướng dẫn, đồng hành và hỗ trợ chúng em trong suốt quá trình thực hiện đề tài "Configure and Deploy MLOps (Machine Learning Ops) Services on Arch Linux & Ubuntu".

Ngay từ những ngày đầu hình thành ý tưởng, thầy đã định hướng rõ ràng cho nhóm về bản chất môn học: tập trung sâu vào kỹ thuật quản trị hệ thống Linux, kiến trúc hạt nhân (Kernel), kỹ thuật gia cố bảo mật chuyên sâu (Security Hardening), và so sánh đối kháng thực nghiệm giữa hai trường phái Linux đối lập: Arch Linux (Rolling Release, DIY) và Ubuntu Server (LTS, Enterprise).

Trong suốt quá trình triển khai – từ việc xây dựng script tự động hóa Bash idempotent, cấu hình tường lửa nftables và UFW, thiết lập cơ chế cô lập Rootless Docker qua User Namespaces và cgroups v2, đến đóng gói microservice AI OCR và giám sát Prometheus/Grafana – nhóm em luôn nhận được những lời khuyên sâu sắc, sự hỗ trợ kỹ thuật và nguồn động viên quý báu từ thầy.

Chúng em cũng xin gửi lời cảm ơn chân thành đến quý Thầy Cô trong Khoa Khoa Học Máy Tính và Hội đồng phản biện Trường Đại học Công nghệ Thông tin và Truyền thông Việt – Hàn đã tạo điều kiện thuận lợi nhất để nhóm hoàn thành và báo cáo đồ án.

Một lần nữa, chúng em kính chúc thầy luôn dồi dào sức khỏe, hạnh phúc và gặt hái thêm nhiều thành công rực rỡ trong sự nghiệp nghiên cứu và giảng dạy.

Chúng em xin chân thành cảm ơn!

NHẬN XÉT

(Của giảng viên hướng dẫn)

.....
.....

.....
.....

.....
.....

.....
.....

.....
.....

.....
.....

.....
.....

.....
.....

.....
.....

.....
.....

.....
.....

.....
.....

.....
.....

.....
.....

.....
.....

.....
.....

Đà Nẵng, ngày .... tháng 10 năm 2026

Chữ ký giảng viên hướng dẫn

TS. Hoàng Hữu Đức

MỤC LỤC

LỜI CẢM ƠN 2

NHẬN XÉT CỦA GIẢNG VIÊN 3

MỤC LỤC 4

DANH MỤC HÌNH ẢNH 6

DANH MỤC BẢNG BIỂU 7

MỞ ĐẦU 8

CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI 10

1.1. Đặt vấn đề 10

1.1.1. Bối cảnh MLOps và nhu cầu chuẩn hóa hạ tầng Linux 10

1.1.2. Thách thức phân phối phần mềm: Rolling Release vs. LTS 12

1.1.3. Lỗ hổng bảo mật đặc quyền Root trên Container Docker truyền thống 13

1.2. Mục tiêu đề tài 14

1.2.1. Mục tiêu tổng quát 14

1.2.2. Các mục tiêu cụ thể 15

1.2.3. Kết quả mong đợi 16

1.3. Đối tượng và phạm vi nghiên cứu 17

1.4. Phương pháp nghiên cứu và thực nghiệm 18

1.5. Ý nghĩa khoa học và thực tiễn của đề tài 19

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ KIẾN TRÚC HỆ THỐNG 21

CHƯƠNG 3: THIẾT KẾ VÀ QUY CHUẨN TRIỂN KHAI IAC 35

CHƯƠNG 4: TRIỂN KHAI VÀ THỰC NGHIỆM ĐỐI KHÁNG 48

CHƯƠNG 5: ĐÁNH GIÁ ĐỊNH LƯỢNG BENCHMARK & BẢO MẬT 62

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 72

TÀI LIỆU THAM KHẢO 76

DANH MỤC HÌNH ẢNH & BẢNG BIỂU

Hình 1.1: So sánh vòng đời phát hành giữa Rolling Release (Arch) và LTS (Ubuntu) 12

Hình 1.2: Mô hình tấn công Container Escape khi chia sẻ chung User Namespace root 14

Hình 2.1: Sơ đồ kiến trúc phân tầng Dual-Node MLOps Platform 22

Hình 2.2: Cơ chế ánh xạ User Namespaces (subuid/subgid) trong Rootless Docker 27

Hình 2.3: Kiến trúc lọc gói tin Stateful Firewall bằng nftables trên Arch Linux 31

Hình 3.1: Sơ đồ luồng phân tách mạng Zero-Trust giữa frontend và backend database 38

Hình 4.1: Log Terminal quá trình cài đặt tự động 01_system_hardening.sh 50

Hình 4.2: Kết quả kiểm tra trạng thái Docker daemon cấp unprivileged user 54

Hình 4.3: Giao diện Terminal tương tác AI OCR Scanner (ai_cli.sh) 58

Hình 5.1: Biểu đồ giám sát thời gian thực CPU và RAM trên Grafana Dashboard 65

Bảng 1.1: Ma trận so sánh tính năng cốt lõi giữa Arch Linux và Ubuntu Server 13

Bảng 2.1: Danh mục dịch vụ MLOps và quy hoạch cổng mạng 24

Bảng 4.1: Danh mục các bài kiểm tra tự động trong 04_verify_system.sh 56

Bảng 5.1: Kết quả Benchmark so sánh thông lượng RPS và độ trễ p95 giữa 2 hệ điều hành 68

Bảng 5.2: So sánh lượng chiếm dụng RAM tĩnh và động (Footprint Analysis) 70

MỞ ĐẦU

Trong làn sóng bùng nổ của Trí tuệ nhân tạo (AI) và Học máy (Machine Learning), việc chuyển giao các mô hình từ môi trường thử nghiệm sang vận hành thực tế đã đặt ra một tiêu chuẩn mới mang tên MLOps (Machine Learning Operations). Tương tự như DevOps trong phát triển phần mềm truyền thống, MLOps đòi hỏi sự kết hợp chặt chẽ giữa mã nguồn, dữ liệu, cấu hình hạ tầng và quy trình tự động hóa. Tuy nhiên, một hệ thống MLOps hiện đại không thể vận hành tách rời khỏi nền tảng cốt lõi của nó: Hệ điều hành Linux.

Linux hiện diện ở hơn 90% các máy chủ tính toán hiệu năng cao, siêu máy tính và trung tâm dữ liệu trên thế giới. Dẫu vậy, thế giới mã nguồn mở Linux không phải là một khối đồng nhất, mà phân hóa sâu sắc thành hai trường phái triết lý phát triển hoàn toàn trái ngược nhau:

• Trường phái Rolling Release (tiêu biểu là Arch Linux): Đại diện cho sự tự do, triết lý tối giản ("The Arch Way"), liên tục cập nhật hạt nhân Linux Kernel và các gói phần mềm lên phiên bản mới nhất ngay khi phát hành. Đây là môi trường lý tưởng cho các kỹ sư muốn tiếp cận công nghệ hạt nhân mới nhất, khả năng tùy biến sâu và kiểm soát tối đa từng byte tài nguyên hệ thống.

• Trường phái Long-Term Support - LTS (tiêu biểu là Ubuntu Server): Đại diện cho tính ổn định, chuẩn mực doanh nghiệp, với chu kỳ phát hành có kế hoạch và cam kết hỗ trợ bảo mật nhiều năm. Ubuntu cung cấp cấu hình mặc định sẵn sàng sử dụng (Out-of-the-box), giảm thiểu chi phí quản trị nhưng chấp nhận sử dụng các phiên bản kernel và gói phần mềm cũ hơn.

Vấn đề đặt ra là: Trong cùng một khối lượng công việc MLOps cụ thể (bao gồm theo dõi thí nghiệm với MLflow, lưu trữ metadata PostgreSQL, lưu trữ artifact S3 MinIO, phục vụ mô hình FastAPI và giám sát Prometheus/Grafana), sự khác biệt về triết lý hệ điều hành, cơ chế bảo mật (tường lửa nftables so với UFW), và đặc biệt là phương thức containerization (Rootless Docker unprivileged so với Docker daemon truyền thống) sẽ tạo ra sự khác biệt như thế nào về độ an toàn và hiệu năng?

Phần lớn các tài liệu kỹ thuật hiện nay chỉ tập trung triển khai container trên một hệ điều hành duy nhất mà bỏ qua việc so sánh đối kháng thực nghiệm giữa hai nền tảng đối lập này. Bên cạnh đó, việc lạm dụng quyền root trong quản trị container Docker đã và đang gây ra các thảm họa bảo mật nghiêm trọng (như lỗ hổng container escape, lộ Docker socket).

Xuất phát từ thực tiễn trên, nhóm nghiên cứu đã lựa chọn thực hiện đề tài: "Configure and Deploy MLOps (Machine Learning Ops) Services on Arch Linux & Ubuntu". Đề tài tập trung xây dựng hai môi trường độc lập hoàn toàn, tự động hóa 100% bằng Infrastructure as Code (Bash Script), áp dụng các tiêu chuẩn an ninh khắt khe nhất của hệ điều hành Linux và đo lường định lượng hiệu năng thực tế.

CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI

1.1. ĐẶT VẤN ĐỀ

1.1.1. Bối cảnh MLOps và nhu cầu chuẩn hóa hạ tầng Linux

MLOps là sự giao thoa giữa Khoa học dữ liệu (Data Science), Kỹ thuật phần mềm (Software Engineering) và Vận hành hệ thống (Operations). Khác với các ứng dụng web thông thường, hệ thống MLOps đòi hỏi sự phối hợp nhịp nhàng giữa nhiều dịch vụ có trạng thái (stateful) và không trạng thái (stateless):

- MLflow Tracking Server: Đóng vai trò máy chủ trung tâm ghi nhận metadata thí nghiệm, thông số siêu dữ liệu và quản lý phiên bản mô hình (Model Registry).

- PostgreSQL Relational Database: Cơ sở dữ liệu quan hệ mạnh mẽ đảm bảo tính toàn vẹn dữ liệu cho MLflow thay thế cho SQLite đơn luồng.

- MinIO Object Storage: Hệ thống lưu trữ đối tượng tương thích chuẩn Amazon S3, tối ưu hóa cho các tệp nhị phân model nặng.

- FastAPI Inference Microservice: Dịch vụ web bất đồng bộ hiệu năng cao phục vụ nhận diện chữ từ hình ảnh (AI OCR Scanner), xuất metrics thời gian thực.

- Prometheus & Grafana: Cụm giám sát chuỗi thời gian (TSDB) và hiển thị trực quan tài nguyên phần cứng máy chủ cùng lưu lượng mạng.

Toàn bộ các thành phần này đòi hỏi một hạ tầng hệ điều hành Linux ổn định, khả năng quản trị tài nguyên CPU/RAM chặt chẽ và cơ chế bảo mật mạng nghiêm ngặt.

1.1.2. Thách thức phân phối phần mềm: Rolling Release vs. LTS

Trong quản trị hệ thống Linux, việc lựa chọn bản phân phối (Linux Distribution) luôn là quyết định mang tính chiến lược của doanh nghiệp:

• Arch Linux (Rolling Release): Không có khái niệm phiên bản cố định. Hệ điều hành liên tục nhận các bản vá hạt nhân Kernel mới nhất từ nhánh phát triển upstream của Linus Torvalds. Người quản trị phải tự tay cấu hình từng thành phần (DIY). Điều này mang lại hiệu năng tối đa và tính năng mới nhất của cgroups v2, nhưng đòi hỏi trình độ chuyên môn cao để đảm bảo hệ thống không bị đổ vỡ.

• Ubuntu Server (LTS): Phát hành định kỳ 2 năm một lần với 5 năm hỗ trợ an ninh. Các gói phần mềm được đóng băng phiên bản để ưu tiên độ ổn định tuyệt đối. Mặc dù dễ vận hành, nhưng hệ thống thường phải dùng các phiên bản kernel cũ và cấu hình mặc định đôi khi chưa được tối ưu hóa cho tải MLOps hiệu năng cao.

1.1.3. Lỗ hổng bảo mật đặc quyền Root trên Container Docker truyền thống

Theo mặc định, Docker daemon chạy dưới quyền root (UID 0) của hệ thống host. Các container bên trong cũng chạy với UID 0. Đây là một lỗ hổng an ninh mang tính hệ

thống:

- Container Escape: Nếu một lỗ hổng thực thi mã từ xa (RCE) xảy ra trong ứng dụng AI (ví dụ: tràn bộ đệm xử lý ảnh), kẻ tấn công chiếm được quyền root trong container và có thể vượt rào thoát ra chiếm toàn quyền kiểm soát máy tính thật (Host OS). Lỗ hổng runc CVE-2019-5736 là bằng chứng tiêu biểu.

- Lộ Docker Socket (/var/run/docker.sock): Bất kỳ người dùng nào có quyền ghi vào socket này đều tương đương nắm giữ quyền root tối cao của máy chủ.

Để giải quyết triệt để vấn đề này, đề tài triển khai công nghệ Rootless Docker trên Arch Linux — nơi Docker daemon và toàn bộ container chạy hoàn toàn dưới một tài khoản người dùng không đặc quyền (non-root), sử dụng User Namespaces (subuid/subgid) để đảm bảo dù container bị chiếm quyền, kẻ tấn công vẫn vô hại trên hệ thống host.

1.2. MỤC TIÊU ĐỀ TÀI

1.2.1. Mục tiêu tổng quát

Thiết kế, tự động hóa và triển khai hoàn chỉnh hai hệ thống MLOps độc lập trên hai bản phân phối Linux đối lập (Arch Linux và Ubuntu Server); thực hiện so sánh đối kháng định lượng về mức độ bảo mật, thông lượng phục vụ API và mức tiêu thụ tài nguyên phần cứng.

1.2.2. Các mục tiêu cụ thể

1. Tự động hóa 100% bằng Infrastructure as Code: Xây dựng bộ script Bash idempotent tuân thủ strict mode (set -euo pipefail), có bẫy lỗi và có script rollback tương ứng cho cả hai hệ điều hành.

2. Gia cố an ninh hệ điều hành (OS Hardening): Đổi cổng SSH sang port 2222, cấm đăng nhập root; triển khai tường lửa gốc nftables có rate-limiting chống brute-force trên Arch Linux và UFW trên Ubuntu Server.

3. Triển khai Rootless Container Isolation: Cấu hình User Namespaces, cgroups v2 resource delegation và systemd lingering để chạy Docker hoàn toàn không cần quyền root trên Arch Linux.

4. Phân tách mạng Zero-Trust: Thiết lập hai mạng Docker bridge riêng biệt; giấu kín hoàn toàn cổng 5432 (Postgres) và cổng S3 nội bộ khỏi giao diện mạng của host.

5. Xây dựng AI Vision Microservice: Đóng gói ứng dụng FastAPI phục vụ nhận diện chữ từ ảnh (OCR Engine) siêu nhẹ, hỗ trợ giao diện Terminal tương tác (CLI) và trích xuất metrics.

6. Đo lường và Đánh giá Thực nghiệm (Benchmarking): Bắn tải đo đạc thông lượng (RPS), độ trễ (Latency) và footprint bộ nhớ RAM giữa Arch Linux và Ubuntu Server.

1.3. ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU

• Đối tượng nghiên cứu: Hệ điều hành Arch Linux x86_64 (Kernel 7.x bleeding-edge), Ubuntu Server 22.04/24.04 LTS (Kernel 5.x/6.x); Docker Engine, RootlessKit; Tường lửa nftables và UFW; Hệ sinh thái MLOps mã nguồn mở (MLflow, PostgreSQL, MinIO, FastAPI, Prometheus, Grafana).

• Phạm vi nghiên cứu: Hai nút tính toán hoạt động độc lập (Dual-node Isolation), không thiết lập cụm giao tiếp mạng giữa hai bên để đảm bảo tính khách quan của phép thử nghiệm đối kháng.

• Giới hạn đề tài: Đề tài tập trung chuyên sâu vào tầng Hạ tầng Hệ thống Linux và Bảo mật Container (System Infrastructure & Container Security), không đi sâu vào việc tối ưu hóa độ chính xác của thuật toán học sâu.

1.4. PHƯƠNG PHÁP NGHIÊN CỨU

Đề tài áp dụng phương pháp nghiên cứu thực nghiệm kết hợp đối sánh định lượng:

1. Phương pháp lý thuyết: Phân tích kiến trúc hạt nhân Linux, cơ chế điều khiển tiến trình cgroups v2, không gian tên User Namespaces và tiêu chuẩn bảo mật CIS Benchmark.

2. Phương pháp thực nghiệm: Lập trình kịch bản Bash tự động triển khai hệ thống trên máy thật Arch Linux và máy ảo Ubuntu Server.

3. Phương pháp đo lường: Sử dụng công cụ curl/hey bắn tải đo thông số hiệu năng và sử dụng lệnh Linux thu thập mức độ sử dụng RAM, CPU.

1.5. Ý NGHĨA KHOA HỌC VÀ THỰC TIỄN

• Ý nghĩa khoa học: Đóng góp một góc nhìn so sánh thực nghiệm toàn diện và chuẩn mực giữa hai mô hình phát hành hệ điều hành Linux (Rolling Release vs. LTS) trong bối cảnh chịu tải MLOps hiện đại.

• Ý nghĩa thực tiễn: Cung cấp một bộ mẫu triển khai (Production-ready Template) hoàn toàn tự động, có thể ứng dụng trực tiếp cho các doanh nghiệp vừa và nhỏ muốn xây dựng nền tảng MLOps an toàn, tiết kiệm chi phí và đạt chuẩn an ninh thông tin cao.